The Collection Hierarchy

8 min read · Updated 28 Jul 2026 · Topic 16 of 65

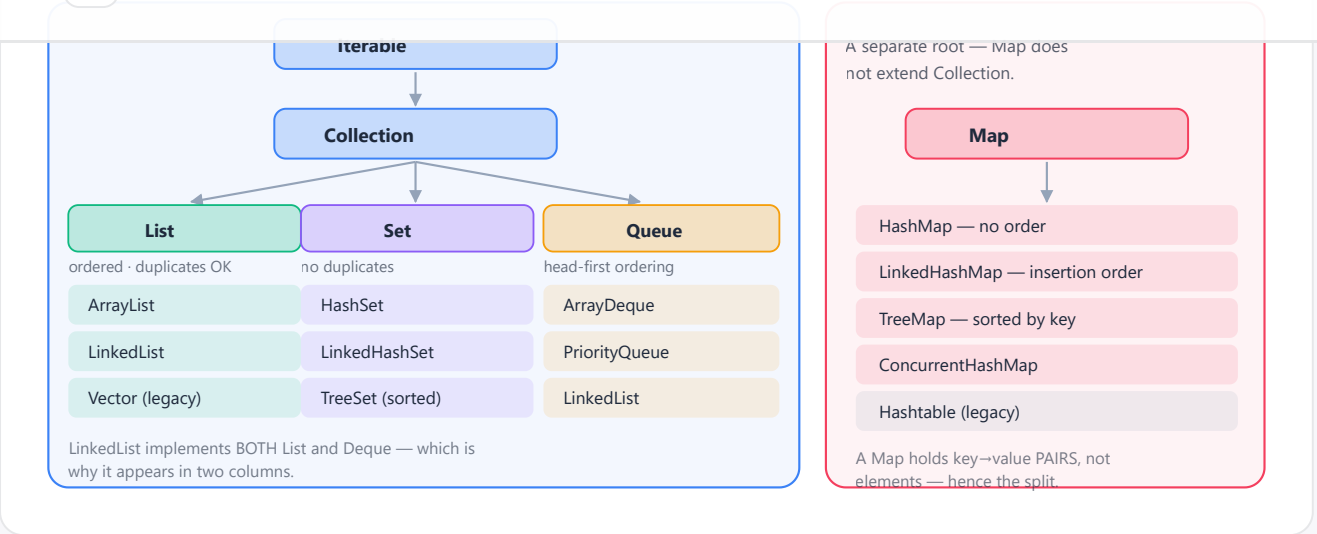
Why it matters

Choosing the wrong collection is one of the few design mistakes that is simultaneously a correctness bug, a performance bug and a readability problem. Interviewers open with the hierarchy because it reveals immediately whether you pick collections deliberately or reach for `ArrayList` and `HashMap` by reflex. It is also the map you need before the six topics that follow.

The concept, simply

The framework has **two independent roots**, and this is the single most commonly missed fact about it:

- `Collection` — a group of individual elements. Its three main sub-interfaces are `List`, `Set` and `Queue`.
- `Map` — a group of key→value *pairs*. `Map` **does not extend** `Collection`, because its element is a pair rather than a value, so methods like `add(E)` would make no sense.



Two roots, not one. Map is deliberately outside the Collection hierarchy — you can get a Collection view of it with keySet(), values() or entrySet(), but a Map is not itself a Collection.

How it works

Each interface is a contract about *semantics*; the implementations differ in the data structure that backs them.

Interface	Guarantees	Reach for
`List`	Ordered by position, duplicates allowed, indexed access	`ArrayList` — nearly
`Set`	No duplicates (by `equals`)	`HashSet`; `LinkedHas`
`Queue` / `Deque`	Ordered for removal — FIFO, LIFO or by priority	`ArrayDeque`; `Prior`
`Map`	Unique keys, one value each	`HashMap`; `LinkedHas`

The naming convention is unusually consistent and worth stating out loud, because it lets you predict behaviour you have never looked up: the **prefix names the data structure** and therefore the guarantees. `Hash*` means a hash table — O(1) average, no order. `Linked*` means a linked list is threaded through it — insertion order preserved, slightly more memory. `Tree*` means a red-black tree — O(log n), sorted, and requires

Program to the interface. Declare variables and parameters as `List`, `Set`, `Map`; use the concrete type only at construction. This lets you change the implementation without touching callers, and it is what interviewers expect to see in your code samples.

Code examples

JAVA Copy

```java
// ✓ Declare the interface, construct the implementation.
List<String> names = new ArrayList<>();
Set<String> unique = new HashSet<>();
Map<String, Integer> counts = new HashMap<>();
Deque<Task> stack = new ArrayDeque<>();

// ✗ Ties every caller to the implementation for no benefit.
ArrayList<String> names = new ArrayList<>();
```

The Map-is-not-a-Collection point, and the three views that bridge the gap:

JAVA Copy

```java
Map<String, Integer> scores = Map.of("asha", 90, "ben", 75);

Set<String> keys          = scores.keySet();     // Set view of the keys
Collection<Integer> vals  = scores.values();     // Collection (values may
Set<Map.Entry<String,Integer>> pairs = scores.entrySet();   // the pairs

// entrySet() is the right way to iterate — one pass, no repeated lookups:
for (Map.Entry<String, Integer> e : scores.entrySet()) {
    System.out.println(e.getKey() + " = " + e.getValue());
}

// ✗ This does a full get() for every key — twice the work:
for (String k : scores.keySet()) System.out.println(k + " = " + scores.get
```

Factory methods worth knowing, and the trap in one of them:

```
// Java 9+ — IMMUTABLE, and the clearest way to write a fixed collection.
List<String> a = List.of("x", "y");         // add() throws UnsupportedOper
Set<String> b  = Set.of("x", "y");          // duplicate arguments throw at
Map<String,Integer> c = Map.of("x", 1);     // rejects null keys AND null v

// Older, and NOT the same thing — a fixed-size view backed by the array:
List<String> d = Arrays.asList("x", "y");
d.set(0, "z");          // ✓ allowed — writes through to the array
// d.add("w");          // ✗ UnsupportedOperationException — size is fixed

// To get a mutable copy from either:
List<String> mutable = new ArrayList<>(List.of("x", "y"));
```

Under the hood

The legacy classes come up constantly, and knowing why they are legacy is a better answer than knowing that they are. `Vector` and `Hashtable` predate the framework (they are from Java 1.0) and synchronise every method. That is the wrong granularity: it costs on every call in the single-threaded case, and it still does not make a *sequence* of operations atomic, which is what concurrent code actually needs. `Stack` is worse — it extends `Vector`, so it inherits `get(int)` and `add(int, E)` and you can reach into the middle of a stack, which defeats the point. Use `ArrayDeque` for a stack and `ConcurrentHashMap` for a concurrent map.

`null` **handling differs by implementation** and it is a favourite question:

Type	null key	null value
`HashMap`	One allowed	Allowed
`TreeMap`	No — must compare	Allowed
`Hashtable`	No	No
`ConcurrentHashMap`	**No**	**No**

`List.of` / `Map.of`	No	No

`ConcurrentHashMap`'s refusal has a good reason worth quoting: in a concurrent map, `get(k)` returning null would be ambiguous — absent, or present-with-a-null-value? In a single-threaded `HashMap` you can disambiguate with `containsKey`, but concurrently the answer could change between the two calls. Banning null removes the ambiguity entirely.

Common mistakes

- **Saying `Map` extends `Collection`.** It does not, and the reason (elements vs pairs) is the interesting half of the answer.
- **Declaring concrete types.** `ArrayList<X> list = ...` couples callers to an implementation choice.
- **Iterating with `keySet()` then `get()`.** Use `entrySet()` — half the lookups.
- **Assuming `Arrays.asList` is immutable.** It is fixed-size but writable, and writes through to the backing array.
- **Calling `add` on a `List.of(...)`.** Genuinely immutable — it throws.
- **Using `Vector`, `Hashtable` or `Stack`.** Legacy. Use `ArrayList`, `ConcurrentHashMap`, `ArrayDeque`.
- **Putting null in a `ConcurrentHashMap`.** Throws `NullPointerException` — by design.

Interview lens

> **Q: "Walk me through the collection hierarchy."**
> A: Two roots. `Collection` extends `Iterable` and splits into `List` (ordered, duplicates, indexed), `Set` (no duplicates) and `Queue` / `Deque` (ordered for removal). `Map` is separate and does not extend `Collection`, because it holds key–value pairs rather than elements — though `keySet()`, `values()` and `entrySet()` give you collection views of it.

A: Because the abstractions do not fit. `Collection`'s contract is built around a single element type — `add(E)`, `contains(E)`, `iterator()` over `E`. A map's unit is a pair, so those methods would either be meaningless or need an arbitrary choice between key and value. Keeping them separate and offering explicit views is cleaner than forcing the inheritance.

Q: "How do you choose between `List`, `Set` and `Map`?"

A: By the question I need to answer. If I need order and duplicates, or index access — `List`. If I need "have I seen this?" with no duplicates — `Set`. If I need "what is associated with this key?" — `Map`. Then the prefix picks the implementation: `Hash` for speed with no order, `Linked` to keep insertion order, `Tree` when I need sorting or range queries.

Q: "`Arrays.asList` vs `List.of`?"

A: `Arrays.asList` returns a fixed-size, mutable view backed by the array — `set` works and writes through, `add` throws, and it permits nulls. `List.of` (Java 9+) returns a genuinely immutable list that rejects nulls and throws on any modification. I use `List.of` unless I specifically need the array view.

Q: "Why can't `ConcurrentHashMap` hold nulls?"

A: Ambiguity that cannot be resolved concurrently. If `get(k)` returned null, you could not tell "absent" from "mapped to null". In a `HashMap` you would call `containsKey` to disambiguate, but in a concurrent map the state can change between the two calls, so the check is worthless. Banning null makes null unambiguously mean absent.

Q: "What is wrong with `Vector` and `Stack`?"

A: Both are Java 1.0 classes that synchronise every method — overhead when you do not need it, and insufficient when you do, since locking each call does not make

replacement and is faster.

Cheat-card

- **Two roots**: `Collection` (List/Set/Queue) and `Map`. Map is **not** a Collection.
- Prefix = structure: **Hash** (O(1), no order) · **Linked** (insertion order) · **Tree** (O(log n), sorted).
- Declare the **interface**, construct the implementation.
- Iterate maps with `entrySet()`, never keySet + get.
- `List.of` = immutable, no nulls. `Arrays.asList` = fixed-size, writable view.
- `HashMap`: 1 null key, null values OK. `ConcurrentHashMap`: **no nulls at all.**
- Legacy: `Vector`, `Hashtable`, `Stack` → use `ArrayList`, `ConcurrentHashMap`, `ArrayDeque`.

QUICK CHECK — no marks, just for you.

1. Why does Map not extend Collection?

- ◯ For historical reasons only — it was an oversight in Java 1.2
- ◯ Because Map is not iterable
- ◯ Because a Map's unit is a key-value PAIR, so Collection's element-based contract — add(E), contains(E), iterator over E — would be meaningless or require an arbitrary choice between key and value
- ◯ Because Map allows duplicate keys

Check answer

2. What is the difference between Arrays.asList("a","b") and List.of("a","b")?

- Arrays.asList is immutable; List.of is mutable
- They are identical
- Arrays.asList is a fixed-size mutable view backed by the array — set works and writes through, add throws — while List.of is fully immutable and rejects nulls

Check answer

3. Why does ConcurrentHashMap reject null keys and values when HashMap allows them?

- Because get() returning null would be ambiguous between 'absent' and 'mapped to null', and unlike a HashMap you cannot resolve it with containsKey — the state may change between the two calls
- To save memory
- It is an arbitrary API restriction with no technical reason
- Because null cannot be hashed

Check answer

Previous
Wrapper Classes, Autoboxing and the Integer Cache

Next
ArrayList vs LinkedList

LEARN MORE

ARTICLES & DOCS

java.util package — API docs
docs.oracle.com

VIDEOS



Java Array vs ArrayList | Collections vs Streams | HashMap vs ConcurrentHashMap | HashMap vs HashSet
Java Guides

Hand-picked references for this lesson.